

OpenCV Practical Codes — Q1 to Q6

Q1 — Basic Image Operations

#1. Convert Color Image to Grayscale

```
import cv2

img = cv2.imread("cat.jpg")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

cv2.imshow("Gray Image", gray)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

#2. Resize an Image

```
import cv2

img = cv2.imread("cat.jpg")

resized = cv2.resize(img, (500, 400))

cv2.imshow("Resized", resized)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

#3. Crop an Image

```
import cv2

img = cv2.imread("cat.jpg")

cropped = img[100:400, 100:500]

cv2.imshow("Cropped", cropped)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

#4. Rotate an Image

```
import cv2

img = cv2.imread("cat.jpg")

rotated = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)

cv2.imshow("Rotated", rotated)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

#5. Flip an Image

```
import cv2

img = cv2.imread("cat.jpg")

flipped = cv2.flip(img, 1)

cv2.imshow("Flipped", flipped)
```

```

cv2.waitKey(0)

cv2.destroyAllWindows()

#6. Access Pixel Values

import cv2

img = cv2.imread("cat.jpg")

pixel = img[100, 100]

print(pixel)

```

```

#7. Change Pixel Values

import cv2

img = cv2.imread("cat.jpg")

img[100,100] = [0,0,255]

cv2.imshow("Modified", img)

cv2.waitKey(0)

cv2.destroyAllWindows()

```

Q2 — Color Space Conversion using OpenCV

```

# colour shape conversion using OpenCV
import cv2

# Read image
img = cv2.imread("cat.jpg")

# Convert image to Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Convert image to HSV
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Display images
cv2.imshow("Original Image", img)
cv2.imshow("Grayscale Image", gray)
cv2.imshow("HSV Image", hsv)

# Print pixel values at (100, 100)
print("RGB Pixel Value:", img[100, 100])
print("Gray Pixel Value:", gray[100, 100])
print("HSV Pixel Value:", hsv[100, 100])

# Wait for a key press and close windows
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Q3 — Brightness, Contrast and Image Filtering

```

# Adjust brightness and contrast

import cv2

# Read image
img = cv2.imread("nature.jpg")

# Adjust contrast and brightness
alpha = 1.5 # Contrast factor
beta = 50 # Brightness value

```

```

result = cv2.convertScaleAbs(img, alpha=alpha, beta=beta)

# Display images
cv2.imshow("Original Image", img)
cv2.imshow("Brightness and Contrast Adjusted", result)

cv2.waitKey(0)
cv2.destroyAllWindows()

# 3.2

import cv2

# Read image
img = cv2.imread("nature.jpg")

# Gaussian Filter
gaussian = cv2.GaussianBlur(img, (5, 5), 0)

# Median Filter
median = cv2.medianBlur(img, 5)

# Display images
cv2.imshow("Original Image", img)
cv2.imshow("Gaussian Filter", gaussian)
cv2.imshow("Median Filter", median)

cv2.waitKey(0)
cv2.destroyAllWindows()

```

Q4 — Histograms

```

# 1. Grayscale Histogram (Intensity Histogram)

import cv2
from matplotlib import pyplot as plt

img = cv2.imread('nature.jpg', 0) # read as grayscale

hist = cv2.calcHist([img], [0], None, [256], [0, 256])

plt.plot(hist, color='black')
plt.title('Grayscale Histogram')
plt.xlabel('Pixel Intensity')
plt.ylabel('Number of Pixels')
plt.show()

# 2. Color Histogram (Per-Channel BGR Histogram)

import cv2
from matplotlib import pyplot as plt

img = cv2.imread('nature.jpg') # colour image (BGR)
colors = ('b', 'g', 'r')

for i, col in enumerate(colors):
    hist = cv2.calcHist([img], [i], None, [256], [0, 256])
    plt.plot(hist, color=col)
    plt.xlim([0, 256])

plt.title('Color Histogram (BGR)')
plt.xlabel('Pixel Intensity')
plt.ylabel('Number of Pixels')
plt.show()

# 3. 2D Histogram (Hue-Saturation Histogram)

import cv2
from matplotlib import pyplot as plt

```

```

img = cv2.imread('nature.jpg')
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

hist2d = cv2.calcHist([hsv], [0, 1], None, [180, 256], [0, 180, 0, 256])

plt.imshow(hist2d, interpolation='nearest')
plt.title('2D Histogram (Hue vs Saturation)')
plt.xlabel('Saturation')
plt.ylabel('Hue')
plt.show()

# 4. Normalized Histogram

hist = cv2.calcHist([img], [0], None, [256], [0, 256])
cv2.normalize(hist, hist, 0, 1, cv2.NORM_MINMAX)

# 5. Cumulative Histogram

import cv2
import numpy as np
from matplotlib import pyplot as plt

img = cv2.imread('nature.jpg')

hist = cv2.calcHist([img], [0], None, [256], [0, 256])
cdf = np.cumsum(hist)

plt.plot(cdf, color='blue')
plt.title('Cumulative Histogram')
plt.xlabel('Pixel Intensity')
plt.ylabel('Cumulative Number of Pixels')
plt.show()

# 6. Equalized Histogram

import cv2

img = cv2.imread('nature.jpg', 0) # grayscale image
equalized = cv2.equalizeHist(img)

cv2.imshow('Original', img)
cv2.imshow('Equalized', equalized)

cv2.waitKey(0)
cv2.destroyAllWindows()

# 7. Histogram with a Mask (Region of Interest)

import numpy as np
import cv2
from matplotlib import pyplot as plt

img = cv2.imread('nature.jpg', 0)

# Create a mask, same size as image
mask = np.zeros(img.shape[:2], dtype='uint8')
mask[100:300, 100:400] = 255 # define region of interest

masked_img = cv2.bitwise_and(img, img, mask=mask)
hist_mask = cv2.calcHist([img], [0], mask, [256], [0, 256])

plt.plot(hist_mask, color='purple')
plt.title('Masked Histogram (ROI only)')
plt.show()

```

Q5 — Sobel and Canny Edge Detection

```
# Implementing Sobel and Canny Edge Detection Techniques using OpenCV
```

```
# 1. Sobel Edge Detection
```

```
# Practical 1: Detect Building Edges
```

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
# Read image
```

```
img = cv2.imread("building.jpg")
```

```
# Convert to grayscale
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Sobel X
```

```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```

```
# Sobel Y
```

```
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
```

```
# Convert to absolute values
```

```
sobel_x = cv2.convertScaleAbs(sobel_x)
```

```
sobel_y = cv2.convertScaleAbs(sobel_y)
```

```
# Combine both directions
```

```
sobel = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)
```

```
# Display
```

```
plt.figure(figsize=(15,5))
```

```
plt.subplot(1,4,1)
```

```
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

```
plt.title("Original")
```

```
plt.axis("off")
```

```
plt.subplot(1,4,2)
```

```
plt.imshow(sobel_x, cmap='gray')
```

```
plt.title("Sobel X")
```

```
plt.axis("off")
```

```
plt.subplot(1,4,3)
```

```
plt.imshow(sobel_y, cmap='gray')
```

```
plt.title("Sobel Y")
```

```
plt.axis("off")
```

```
plt.subplot(1,4,4)
```

```
plt.imshow(sobel, cmap='gray')
```

```
plt.title("Combined Sobel")
```

```
plt.axis("off")
```

```
plt.show()
```

```
# 2. Canny Edge Detection
```

```
# Practical 2: Road Lane Detection (Real-Life Example)
```

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
# Read image
```

```
img = cv2.imread("road.jpg")
```

```
# Convert to grayscale
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Remove noise
```

```
blur = cv2.GaussianBlur(gray, (5,5), 1.4) #(src, kernel, sigma)
```

```
# Canny edge detection
```

```
edges = cv2.Canny(blur, 50, 150) # (image, threshold1, threshold2)
```

```
# Display
```

```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Road")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis("off")

plt.show()
```

Q6 — Harris Corner Detection

Detect Corners using Harris Corner Detector

```
import cv2
import numpy as np

# Read image
image = cv2.imread("building.jpg")

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Convert grayscale image to float32
gray = np.float32(gray)

# Apply Harris Corner Detector (src, blockSize2x2, ksize3x3, k)
corners = cv2.cornerHarris(gray, 2, 3, 0.04)

# Dilate the corner points(darker edge)
corners = cv2.dilate(corners, None)

# Mark detected corners in red
image[corners > 0.01 * corners.max()] = [0, 0, 255]

# Display result
cv2.imshow("Harris Corner Detection", image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```